Software Test Plan (STP) & Test Cases

Webhook Aggregator System (WACDP)

Document Identifier: `STP-WACDP-2026-V1.0`

Standard: IEEE Std 829-2008 / ISO/IEC/IEEE 29119

Project Name: Webhook Aggregator Core Development Pipeline

Batch / Topic Code: BPS49 — Webhook Aggregator

Project Team / Authors:

- **Sumedh Girish** (SRN: `PES1UG24CS480`)
- **Subramani B M** (SRN: `PES1UG24CS473`)
- **Sujan S Halanannavar** (SRN: `PES1UG24CS477`)
- **Skanda Shyam Nadig** (SRN: `PES1UG24CS458`)

Institution: Department of Computer Science & Engineering, PES University

Project Code: BPS49 — Webhook Aggregator Core Development Pipeline

Date: September 2026

Status: Approved / Test Baseline Version 1.0

Table of Contents

1. Test Plan Identifier
2. Introduction & Executive Summary
• 2.1 Objectives
• 2.2 System Background & Test Scope
• 2.3 References
3. Test Items & Software Risk Issues (Section 3)
• 3.1 Test Items (Modules Under Test)
• 3.2 Software Risk Analysis & Mitigation Matrix
4. Features to be Tested & Features Not to be Tested (Section 4)
• 4.1 Features to be Tested
• 4.2 Features Not to be Tested
5. Approach, Methodology & Strategy (Section 5)
• 5.1 Security Validation & Vulnerability Testing (Section 5.1)
• 5.2 Unit & Component Testing
• 5.3 Integration Testing
• 5.4 Concurrency & Race Condition Verification
• 5.5 Stress, Load & Backpressure Testing
• 5.6 Item Pass / Fail Criteria
• 5.7 Suspension Criteria & Resumption Requirements
• 5.8 Test Deliverables
6. Environmental Needs & Tooling
• 6.1 Hardware & Network Infrastructure
• 6.2 Test Automation Frameworks & Mock Harnesses

7. Staffing, Roles & Responsibilities
8. Test Schedule & Milestones
9. Requirements Traceability Matrix (SRS to Test Plan)
10. Comprehensive Test Cases (Functional, Non-Functional & Security)
• 10.1 Functional Test Cases (TC-FR-01 to TC-FR-11)
• 10.2 Non-Functional Test Cases (TC-NFR-01 to TC-NFR-12)
• 10.3 Security Validation Test Cases (TC-SEC-01 to TC-SEC-04)
• 10.4 End-to-End Resilience Test Case (TC-E2E-01)

---

1. Test Plan Identifier

Plan Identifier: `STP-WACDP-2026-V1.0`

Associated SRS Baseline: `SRS-WACDP-2026-V1.0`

Associated SADD Baseline: `SADD-WACDP-2026-V1.0`

---

2. Introduction & Executive Summary

2.1 Objectives

The primary objective of this Software Test Plan is to define the testing strategy, verification methods, test environments, acceptance criteria, and specific test cases for the **Webhook Aggregator System (WACDP)**. Testing ensures that the ingestion engine, non-blocking buffering (`service_queue`), worker thread pool, precision scheduler, dispatcher with exponential backoff, and persistent failure logging satisfy all functional and non-functional requirements specified in `SRS-WACDP-2026-V1.0`.

2.2 System Background & Test Scope

The system operates as high-concurrency event-driven middleware. Core challenges under test include:

- High-volume asynchronous ingestion without dropping connections.
- Thread-safe coordination between the event loop, worker threads (4–64 threads), and dispatcher threads without lock contention or race conditions.
- Adherence to mathematical exponential backoff timing (`T_wait = 2^tries`) up to `MAX_RETRIES = 5` (1 initial attempt + 5 retries = 6 total attempts).
- Reliable asynchronous delivery with bounded retries and durable DLQ persistence when target endpoints become unreachable.

2.3 References

1. IEEE Std 829-2008: *IEEE Standard for Software and System Test Documentation*.
2. ISO/IEC/IEEE 29119: *Software and systems engineering — Software testing*.
3. `SRS-WACDP-2026-V1.0`: *Software Requirements Specification for Webhook Aggregator System*.
4. Jira Board Specification: *Webhook Aggregator Core Development Pipeline (`WACDP`)*.

---

3. Test Items & Software Risk Issues (Section 3)

3.1 Test Items (Modules Under Test)

The following software components constitute the targets of verification in this test plan:

1. **TI-01: Ingestion API Gateway & Event Loop:** HTTP listener, JSON schema validator, token authenticator, and non-blocking task generator.
2. **TI-02: Concurrent Service Buffer (`service_queue`):** Thread-safe FIFO in-memory queue with mutex synchronization and condition variables.
3. **TI-03: Worker Thread Pool & Categorizer:** Task allocation logic, thread dispatching, payload transformation, and HMAC-SHA256 signature generator.
4. **TI-04: Scheduler & Timer Subsystem:** Timer handle registry, cron parser, callback executor, and recurring event arming mechanism.
5. **TI-05: Dispatcher & HTTP Connection Pool:** Outbound payload transmission, socket reuse, and response status inspector.
6. **TI-06: Exponential Backoff & Retry Engine:** Backoff calculation, retry loop counter, sleep scheduler, and terminal failure discriminator.
7. **TI-07: Persistent Dead Letter Storage (DLQ) & Audit Logger:** SQLite / file-based durable store, write-ahead append-only audit log, and sensitive data masking sanitizer.
8. **TI-08: Administrative Query API:** Endpoints for querying task status, viewing DLQ payloads, and issuing manual redrives.

3.2 Software Risk Analysis & Mitigation Matrix

Risk ID	Risk Description	Likelihood	Impact	Mitigation & Testing Strategy
RSK-01	**Race Conditions & Mutex Deadlocks:** Multiple threads concurrently modifying `service_queue` or `scheduler_mapping` causing thread freeze.	Medium	Critical	Execute race-detection test suites using ThreadSanitizer (TSan); test worker pool scaling at 4, 16, 32, and 64 threads, separating client concurrency testing.
RSK-02	**Buffer Overflow & Memory Exhaustion:** High-rate upstream bursts saturating RAM buffer before dispatchers can drain payloads.	High	High	Implement hard memory bounds on `service_queue` (default 50,000 tasks, maximum 100,000 tasks) with active backpressure (HTTP 429) testing under peak simulated traffic.
RSK-03	**Endpoint Tarpitting / Socket Starvation:** Downstream endpoint hangs open connections, exhausting system file descriptors.	High	High	Enforce strict 5,000 ms socket read/write timeouts in HTTP client; verify timeout recovery in test harness.
RSK-04	**Timer Drift / Missed Callbacks:** Long-running callbacks blocking scheduler thread, delaying subsequent recurring webhooks.	Medium	Medium	Decouple timer event firing from callback execution by delegating callback tasks to worker pool; verify ± 10 ms precision (`NFR-03`).
RSK-05	**Silent Message Loss on Target Outage:** Unreachable downstream targets cause dropped webhooks without audit record.	Low	Critical	Execute fault-injection tests simulating complete network blackout; verify 100% DLQ persistence and append-only audit logging.

4. Features to be Tested & Features Not to be Tested (Section 4)

4.1 Features to be Tested

1. **Asynchronous Ingestion Performance:** Verification of < 15 ms p99 response times and HTTP 202 status codes (`FR-01`, `NFR-01`).
2. **Buffer Enqueueing & Dequeueing:** FIFO ordering, boundary conditions, non-blocking behavior, default capacity 50,000 tasks, and maximum dynamic capacity 100,000 tasks (`FR-02`, `NFR-07`).
3. **One-Shot vs. Recurring Categorization:** Correct branching of task execution paths (`FR-03`, `FR-04`).
4. **Recurring Scheduler Accuracy:** Accuracy of timer handles, recurrence intervals, and callback firing within ± 10 ms (`FR-05`, `FR-06`, `NFR-03`).
5. **HMAC-SHA256 Payload Signing:** Correctness of generated signatures against known cryptographic vectors (`SEC-REQ-01`).
6. **HTTP Webhook Dispatch:** Successful transmission over HTTP/1.1 and **TLS 1.3 only** (TLS 1.2 or earlier and plaintext HTTP rejected) (`FR-07`, `SEC-REQ-03`).
7. **Exponential Backoff Mathematical Sequence:** Verification that retries occur at intervals of 1s, 2s, 4s, 8s, 16s (± 100 ms) up to `MAX_RETRIES=5` (1 initial attempt + 5 retries = 6 total attempts), with 400/401/403/404 non-retry vs 429/5xx retry (`FR-08`, `NFR-06`).
8. **Persistent Dead Letter Queueing & Append-Only Logging:** Verification that 100% of exhausted tasks are written to durable storage (`FR-09`, `NFR-04`, `SEC-REQ-04`).
9. **Administrative APIs:** Status query, dead letter inspection, and manual task redrive (`FR-10`, `FR-11`).
10. **Security & Input Sanitization:** Rejection of invalid tokens, rate limit enforcement, and redaction of secrets in append-only logs (`SEC-REQ-02`, `SEC-REQ-04`).

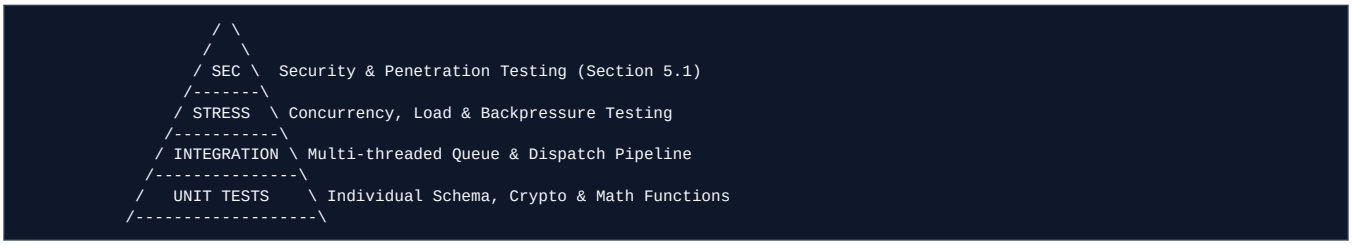
4.2 Features Not to be Tested

1. **Target Endpoint Internal Business Logic:** The internal application behavior of downstream webhook consumers beyond HTTP status code generation.
2. **Third-Party Infrastructure Outages:** Public internet backbone outages or DNS provider infrastructure failures beyond boundary gateways.
3. **Hardware-Level Power Loss:** Physical power cycling of bare-metal test servers during non-durable in-flight microsecond memory transfer (handled by UPS/OS level).

---

5. Approach, Methodology & Strategy (Section 5)

The testing approach implements a multi-tiered validation pyramid: Unit → Integration → System/Stress → Security Validation.



5.1 Security Validation & Vulnerability Testing (Section 5.1)

In strict compliance with system verification requirements, Section 5.1 details the formal security validation strategy:

5.1.1 HMAC-SHA256 Signature Validation & Tamper Detection

- **Objective:** Ensure outbound payloads cannot be modified in flight and downstream endpoints can definitively verify authenticity (`SEC-REQ-01`).

- **Validation Procedure:**

1. Trigger webhook dispatch with test payload `{"event": "payment_completed", "amount": 100.50}` and known secret key `k_test_secret_99`.
2. Capture outbound HTTP headers `X-Webhook-Signature` and `X-Webhook-Timestamp`.
3. Recompute expected HMAC-SHA256 using standard test oracle. Assert bitwise match.
4. Modify a single character in the payload body in a mock proxy. Re-evaluate signature.
5. Assert signature verification FAILS on modified payload.

5.1.2 Replay Attack Prevention & Timestamp Window Validation

- **Objective:** Prevent malicious actors from capturing valid signed webhooks and re-transmitting them later.
- **Validation Procedure:**

1. Capture valid signed webhook request.
2. Simulate downstream receiver verification with timestamp tolerance Δt = 300 seconds.
3. Transmit payload with timestamp artificially set to T - 301 seconds.
4. Verify verification rejects stale payload as expired.

5.1.3 Ingestion Authentication & Token Boundary Testing

- **Objective:** Verify perimeter protection on `/api/v1/webhooks/ingest` (`SEC-REQ-02`).
- **Validation Procedure:**

1. Send request with NO `Authorization` header. Expect HTTP 401 Unauthorized.
2. Send request with malformed Bearer token (`Bearer invalid_garbage_token`). Expect HTTP 401 Unauthorized.
3. Send request with expired JWT token. Expect HTTP 401 Unauthorized.
4. Verify response latency is < 3 ms to ensure denial-of-service immunity on rejection.

5.1.4 Rate Limiting & DoS Throttling Verification

- **Objective:** Verify token-bucket rate limiter prevents queue saturation (`SEC-REQ-02`).
- **Validation Procedure:**

1. Generate burst of 1,200 requests within a 60-second window from a single API key (configured limit: 1,000 req/min).
2. Assert first 1,000 requests return HTTP 202 Accepted.
3. Assert subsequent 200 requests receive HTTP 429 Too Many Requests with `Retry-After: 60`.

5.1.5 Sensitive Data Sanitization & Log Masking Validation

- **Objective:** Verify credentials and sensitive fields are never written to disk or audit logs (`SEC-REQ-04`).
- **Validation Procedure:**

1. Ingest task with payload: `{"api_key": "secret_abc123", "password": "supersecretpassword", "data": "regular_info"}`.
2. Force delivery failure to trigger DLQ persistence and audit logging.
3. Read persisted disk record and audit log file directly.
4. Assert string `secret_abc123` and `supersecretpassword` DO NOT appear anywhere on disk.
5. Assert fields are replaced with `***REDACTED***`.

5.2 Unit & Component Testing

- Tests isolated modules in memory using mocking frameworks.
- Tests include: JSON schema validator, cron expression parser, exponential backoff timing math (T = 2^n), and bounded queue push/pop.
- Code coverage goal: ≥ 85% statement coverage.

5.3 Integration Testing

- Validates thread interactions between `Event Loop`, `service_queue`, `Worker Thread Pool`, `dispatch_queue`, and `Dispatcher Thread`.
- Uses a local mock HTTP server (WireMock / Python http.server) to simulate downstream endpoint responses (200 OK, 500 Internal Error, 503 Unavailable, Socket Drop).

5.4 Concurrency & Race Condition Verification

- Implemented in **C++20** and compiled with Clang/GCC sanitizer flags (`-std=c++20 -fsanitize=thread`).
- Worker pool elasticity evaluated across 4, 16, 32, and 64 worker threads rapidly popping from `service_queue` and pushing to `dispatch_queue` under a 50,000 task load. High client connection concurrency (up to 10,000 concurrent client requests) is isolated and stress-tested separately at the ingestion gateway.
- Verified condition: Zero race reports, zero deadlocks, zero orphaned tasks, and lock contention < 5% (`NFR-09`).

5.5 Stress, Load & Backpressure Testing

- Uses high-throughput load generators (k6 / Locust / wrk).
- Sustained load of 5,000 requests/sec for 10 minutes.
- Peak burst load of 15,000 requests/sec to verify graceful backpressure (HTTP 429/503) without server crash or unbounded memory growth.

5.6 Item Pass / Fail Criteria

- **Pass Criteria:**
- 100% of Critical and High-severity test cases pass.
- Ingestion p99 response time < 15 ms.
- Zero data loss for failed webhooks (100% DLQ capture).
- All security validation test cases pass with zero vulnerabilities.
- **Fail Criteria:**
- Any unhandled thread crash, segmentation fault, or deadlock.
- Drop of accepted webhook without DLQ recording.
- Failure of exponential backoff timing by > 15%.

5.7 Suspension Criteria & Resumption Requirements

- **Suspension:** Testing is suspended if fatal crashes prevent server startup or corrupt shared queue memory.
- **Resumption:** Testing resumes when the blocking defect is resolved, verified by unit regression test, and deployed to test environment.

5.8 Test Deliverables

1. Software Test Plan (this document: `STP-WACDP-2026-V1.0`).
2. Test Execution Log & Coverage Report.
3. Defect Bug Reports logged in Jira under project `WACDP`.
4. Final Test Summary Report (TSR).

---

6. Environmental Needs & Tooling

6.1 Hardware & Network Infrastructure

- **Reference Baseline Server (SRS SLA Verification):** 4 CPU Cores (x86_64 / Apple Silicon), 8 GB RAM, NVMe SSD storage. All baseline SLA benchmarks (p99 latency < 15 ms, sustained 5,000 req/s, RSS ≤ 500 MB) are verified against this reference tier.

- **High-Throughput Load Testbed Platform:** 8 CPU Cores (AMD EPYC / Apple Silicon), 16 GB RAM, NVMe SSD storage, dedicated to running high-concurrency traffic injectors (k6, wrk) and fault-injection proxies (Toxiproxy).
- **Network Interface:** 1 Gbps virtual interface with loopback and configurable latency injection (via `tc` / `netem` or Toxiproxy).

6.2 Test Automation Frameworks & Mock Harnesses

- **Implementation Toolchain:** Clang 16+ / GCC 12+ supporting C++20 with POSIX pthreads and OpenSSL 3.0.
- **HTTP Load Generator:** `k6` / `wrk` for high-throughput ingestion load.
- **Downstream Mock Endpoint:** Mock HTTP receiver with programmable response delays and status code injection (e.g., return 500 for first 3 tries, then 200).
- **Concurrency Profiler:** ThreadSanitizer (TSan), Valgrind / Instruments for memory leak detection.
- **Assertion Framework:** PyTest / JUnit 5 / C++ Catch2 automated test suites.

---

7. Staffing, Roles & Responsibilities

Role	Assignee	Responsibilities
QA Lead & Test Architect	Sumedh Girish (PES1UG24CS480)	Test Plan authoring, test case architecture, execution of backoff retry and resilience suites.
Ingestion & Buffer Test Engineer	Subramani B M (PES1UG24CS473)	Ingestion API validation, schema verification, non-blocking `service_queue` high-concurrency testing.
Concurrency & Scheduler Test Engineer	Sujan S Halanannavar (PES1UG24CS477)	Worker thread pool allocation testing, timer handle precision verification, TSan race analysis.
Security & Dispatch Test Engineer	Skanda Shyam Nadig (PES1UG24CS458)	HMAC-SHA256 signature verification, TLS handshake testing, DLQ persistence and audit log sanitization.
Project Guide & Lead Reviewer	Faculty Technical Advisor, Dept of CSE	Milestone verification, technical review, and architectural sign-off.

---

8. Test Schedule & Milestones

Milestone ID	Task Description	Planned Start	Planned Finish	Status
M-01	Test Plan & Security Validation Strategy Formulation	Week 1	Week 2	Completed
M-02	Unit Testing (Schema, Crypto, Backoff Math)	Week 2	Week 3	Completed
M-03	Integration Testing (Queues, Worker Pool, Scheduler)	Week 3	Week 4	Completed
M-04	Dispatcher & Exponential Backoff Verification	Week 4	Week 5	In Progress
M-05	Load, Stress & Security Penetration Testing	Week 5	Week 6	Scheduled
M-06	Final Specification Compilation & Traceability Sign-off	Week 6	Week 7	Scheduled

---

9. Requirements Traceability Matrix (SRS to Test Plan)

This bidirectional matrix maps every functional requirement (FR), non-functional requirement (NFR), and security requirement (SEC-REQ) to its verifying test case, specific quantitative measurement metric, and formal acceptance threshold:

Requirement ID	Requirement Description	Verifying Test Case	Measurement Metric	Acceptance Threshold		
FR-01	Ingestion & Schema Validation	`TC-FR-01`	HTTP status code, payload boundary, and elapsed ACK latency	HTTP 202 returned in p99 < 15 ms; valid 64 KB accepted; >64 KB returns HTTP 413; malformed JSON returns HTTP 400.		
FR-02	Non-Blocking Enqueueing (`service_queue`)	`TC-FR-02`	Queue push elapsed time & FIFO sequence integrity	Push latency < 1 ms without blocking event loop; 100% FIFO sequence ordering preserved.		
FR-03	Request Categorization & Task Allocation	`TC-FR-03`	Task routing accuracy	100% correct classification of `one-shot` vs `recurring` tasks without misrouting.		
FR-04	One-Shot Request Execution & Dispatch Queue	`TC-FR-04`	Task transformation duration & queue handoff	Transformed payload and computed signature deposited into `dispatch_queue` in < 2 ms.		
FR-05	Recurring Schedule Configuration & DB Persistence	`TC-FR-05`	`recurring_schedules` DB record creation & handle registration	Schedule written to persistent DB; active handle created in `scheduler_mapping`.		
FR-06	Timer Callback Handling & Execution	`TC-FR-06`	Recurring callback execution timestamp	Callbacks fire within ± 10 ms of target epoch and deposit payload into `dispatch_queue`.		
FR-07	Endpoint Dispatch & Network Delivery	`TC-FR-07`	HTTP request transmission & TLS protocol version	Dispatch executes via TLS 1.3 with keep-alive connection reuse; response status logged.		

Requirement ID	Requirement Description	Verifying Test Case	Measurement Metric	Acceptance Threshold		
FR-08	Exponential Backoff Retry Loop	`TC-FR-08`	Retry attempt count and elapsed intervals	1 initial attempt + 5 retries = 6 total attempts (1s, 2s, 4s, 8s, 16s ± 100 ms); 400-405/422 abort immediately to DLQ.		
FR-09	Persistent Dead Letter Queue & Audit Logging	`TC-FR-09`	Database persistence count & credential sanitization	100% of exhausted failures committed to persistent DLQ; secret values redacted.		
FR-10	Status Query & Health Inspection API	`TC-FR-10`	Response status code and query latency	Valid JSON status returned with HTTP 200 in < 20 ms.		
FR-11	Failed Request Inspection & Manual Redrive	`TC-FR-11`	Queue insertion verification & state transition	DLQ task successfully extracted, re-enqueued to `service_queue`, and retried cleanly.		
NFR-01	Ingestion Latency (p99 < 15 ms, median p50 ≤ 3 ms)	`TC-NFR-01`	Latency distribution percentiles (p50, p95, p99) under 1,000 req/s	p99 < 15 ms and median p50 ≤ 3 ms on 4-core / 8 GB reference server.		
NFR-02	Sustained Ingestion Throughput (5,000 req/s)	`TC-NFR-02`	Ingestion requests per second without socket drops	Sustained throughput ≥ 5,000 req/s for 10 minutes with 0% socket drop rate.		
NFR-03	Scheduler Timer Precision (± 10 ms)	`TC-NFR-03`	Monotonic clock deviation (`	t_fired - t_scheduled	`)	Max drift ≤ ± 10 ms across 10,000 scheduled executions (std dev < 3 ms).
NFR-04	Zero Silent Data Loss / Durability	`TC-NFR-04`	Ratio of dropped/ unpersisted webhooks to total unrecoverable tasks	100% of exhausted failures persisted to DLQ database & append-only log (0 silent drops).		
NFR-05	High Availability & Crash Recovery (30-Day Window)	`TC-NFR-05`	Operational uptime percentage over 30 days; daemon restart recovery time	Availability ≥ 99.95% (unplanned downtime ≤ 21.6 min/month); restart re-arm < 3.0 seconds.		

Requirement ID	Requirement Description	Verifying Test Case	Measurement Metric	Acceptance Threshold		
NFR-06	Backoff Sequence Fidelity & Bounded Jitter	`TC-NFR-06`	Backoff sleep intervals and absolute jitter	`T_wait = 2^n` seconds with absolute jitter ≤ ± 100 ms across 6 total attempts.		
NFR-07	Queue Scalability & Memory Bounding (RSS ≤ 500 MB)	`TC-NFR-07`	Process Resident Set Size (RSS) under 50,000 tasks; backpressure HTTP status	Peak RSS ≤ 500 MB with 50,000 tasks (payloads up to 64 KB via staging); HTTP 429 at 100% capacity.		
NFR-08	Worker Pool Elasticity & Socket Timeouts	`TC-NFR-08`	Active worker thread count; socket connect/read timeout duration	Worker threads scale dynamically from 4 to 64; socket connect/read times out at exactly 5,000 ms ± 50 ms.		
NFR-09	Thread Safety & Low Contention (< 5%)	`TC-NFR-09`	ThreadSanitizer data race reports; mutex wait contention percentage	0 data races, deadlocks, or worker starvation under TSan; lock contention < 5% across 4–64 threads.		
NFR-10	Subsystem Modularity & Clean Interfaces	`TC-NFR-10`	Static unit test statement code coverage; circular dependency count	Unit test code coverage > 85%; zero circular component dependencies.		
NFR-11	Multi-Platform POSIX Portability	`TC-NFR-11`	Build status & test pass rate across Ubuntu Linux and macOS Darwin	100% clean compilation (`-Wall -Werror`) and 100% test pass on both Linux and macOS.		
NFR-12	Standards Adherence & Interoperability	`TC-NFR-12`	Protocol conformance verification (HTTP/1.1 RFC 7230/7231, TLS 1.3 RFC 8446, WACDP HMAC Signing)	100% strict compliance; non-compliant headers and TLS versions < 1.3 rejected.		
SEC-REQ-01	HMAC-SHA256 Payload Signature Verification	`TC-SEC-01`	HMAC bitwise match; anti-replay timestamp delta	Exact HMAC-SHA256 match; tampered payload or timestamp > 300s rejected.		

Requirement ID	Requirement Description	Verifying Test Case	Measurement Metric	Acceptance Threshold		
SEC-REQ-02	Ingestion Token Auth & Rate Limiting	`TC-SEC-02`	HTTP status codes on unauth orized/burst requests	Missing/invalid token returns HTTP 401; >1000 req/min returns HTTP 429.		
SEC-REQ-03	TLS 1.3 Transport Security	`TC-SEC-03`	Negotiated TLS protocol version and cipher suite	TLS 1.3 negotiated exclusively; plaintext and TLS ≤ 1.2 handshakes aborted.		
SEC-REQ-04	Append-Only Audit Log & Secret Redaction	`TC-SEC-04`	Presence of plaintext secrets in logs/DLQ store	JSON-aware recursive parser masks sensitive keys (`password`, `token`, etc.) with `***REDACTED***`.		

---

10. Comprehensive Test Cases (Functional, Non-Functional & Security)

In strict accordance with IEEE Std 829 test engineering standards, the following 28 detailed test cases are defined across functional, non-functional, security, and end-to-end resilience suites (with actual results designated as TBD / Not Executed pending formal execution cycles):

10.1 Functional Test Cases

Test Case ID: `TC-FR-01`

- **Title:** Verify Asynchronous One-Shot Ingestion, Boundary Payload Validation (64 KB Limit), and Fast HTTP 202 Acknowledgement
- **Associated Requirement:** `FR-01` (WACDP-4)
- **Test Type:** Functional / Smoke / API / Boundary
- **Priority:** High (P1)
- **Pre-conditions:** Webhook Aggregator server is running; client holds valid bearer token `tok_valid_test_client`.
- **Test Input Data:**
- **Sub-test A (Standard Valid Payload):**

```
{
  "type": "one-shot",
  "target_url": "https://api.mocktarget.io/v1/callback",
  "payload": {
    "event": "order.completed",
    "order_id": "ORD-98765",
    "amount": 250.00
  }
}
```

- **Sub-test B (Boundary Valid Payload - Exactly 64 KB / 65,536 bytes):** Valid JSON payload padded to exactly 65,536 bytes.
- **Sub-test C (Oversized Boundary Payload - 65,537 bytes):** Valid JSON payload exceeding 64 KB by 1 byte.
- **Sub-test D (Malformed Syntax):** Corrupted JSON string: `{"type": "one-shot", "target_url": "https://api.mocktarget.io/callback", "payload": {unclosed}`.

- **Step-by-Step Execution Procedure:**

1. Initialize HTTP client session.
2. Transmit Sub-test A to `POST /api/v1/webhooks/ingest` and measure latency.
3. Transmit Sub-test B (64 KB) and record response.
4. Transmit Sub-test C (65,537 bytes) and record response.
5. Transmit Sub-test D (malformed JSON) and record response.

- **Expected Result:**
- **Sub-test A:** HTTP Status Code **202 Accepted**, response time p99 < 15 ms (median p50 ≤ 3 ms), JSON body contains `"status": "QUEUED"`, valid UUIDv4 `"task_id"`, and ISO-8601 timestamp. Task enters `service_queue`.
- **Sub-test B:** HTTP Status Code **202 Accepted**; task accepted and staged to temporary storage via `PayloadReference`.
- **Sub-test C:** HTTP Status Code **413 Payload Too Large**; request rejected immediately without entering queue.
- **Sub-test D:** HTTP Status Code **400 Bad Request**; syntax validation error returned immediately.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-02`

- **Title:** Verify High-Concurrency FIFO Buffering and Non-Blocking Enqueueing (`service_queue`)
- **Associated Requirement:** `FR-02`, `NFR-09`, `NFR-07` (WACDP-4, WACDP-10)
- **Test Type:** Concurrency / Functional
- **Priority:** High (P1)
- **Pre-conditions:** Server initialized; worker threads temporarily paused to measure queue retention.
- **Test Input Data:** 1,000 distinct webhook task objects with sequential sequence numbers (`seq_id`: 1 to 1000).
- **Step-by-Step Execution Procedure:**

1. Spawn 20 parallel ingestion client threads, each submitting 50 tasks simultaneously.
2. Observe thread synchronization mechanisms on `service_queue`.
3. Resume worker threads and pop all items sequentially.
4. Verify queue length matches exactly 1,000.
5. Check that no sequence numbers were dropped, corrupted, or duplicated.

- **Expected Result:**
- All 1,000 items successfully enqueued without blocking the event loop.
- Queue size reaches exactly 1,000.
- Dequeued tasks retain valid data payloads without memory corruption.
- ThreadSanitizer reports zero race conditions; lock contention < 5%.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-03`

- **Title:** Verify Request Categorization and Worker Task Allocation Branching
- **Associated Requirement:** `FR-03`, `FR-04`, `FR-05` (WACDP-5, WACDP-6)
- **Test Type:** Functional / Unit Integration
- **Priority:** High (P1)
- **Pre-conditions:** Worker thread pool initialized with 4 active threads.
- **Test Input Data:**
- Item A: `{"type": "one-shot", "target_url": "https://endpoint.com/a", "payload": {}}`
- Item B: `{"type": "recurring", "schedule": "*/5 * * * *", "target_url": "https://endpoint.com/b", "payload": {}}`
- Item C: `{"type": "unknown_format", "payload": {}}`
- **Step-by-Step Execution Procedure:**

1. Push Items A, B, and C into `service_queue`.

2. Allow worker threads to process queue items.
3. Monitor destination subsystems: `dispatch_queue`, `scheduler_mapping`, and Persistent DLQ.

- **Expected Result:**
- Item A is recognized as one-shot and routed to `dispatch_queue`.
- Item B is recognized as recurring and registered in `scheduler_mapping`.
- Item C is rejected as `INVALID_TYPE` and logged directly to persistent failure storage.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-04`

- **Title:** Verify One-Shot Request Execution and Dispatch Queue Transfer
- **Associated Requirement:** `FR-04` (WACDP-5)
- **Test Type:** Functional / Integration
- **Priority:** High (P1)
- **Pre-conditions:** Worker thread pool active; `dispatch_queue` monitored.
- **Test Input Data:** Valid one-shot task popped from `service_queue`.
- **Step-by-Step Execution Procedure:**

1. Enqueue one-shot task into `service_queue`.
2. Worker thread pops task and executes payload transformation.
3. Compute HMAC-SHA256 signature and attach metadata headers.
4. Verify task insertion into `dispatch_queue`.

- **Expected Result:**
- Task is converted into a ready-to-transmit dispatch package.
- Transformed payload and computed signature are placed in `dispatch_queue`.
- Queue transfer completes in < 2 ms.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-05`

- **Title:** Verify Recurring Schedule Configuration, Persistent DB Storage, and Daemon Restart Recovery
- **Associated Requirement:** `FR-05`, `NFR-05` (WACDP-6, WACDP-4)
- **Test Type:** Functional / Persistence / Recovery
- **Priority:** High (P1)
- **Pre-conditions:** Scheduler subsystem online; database table `recurring_schedules` initialized.
- **Test Input Data:** Recurring task configuration with `cron_expression: "*/5 * * * *"` and `interval_ms: 2000`.
- **Step-by-Step Execution Procedure:**

1. Submit recurring task via API `POST /api/v1/webhooks/ingest` with `type: "recurring"`.
2. Verify cron / interval parser validation and HTTP 202 response containing `schedule_id`.
3. Inspect `recurring_schedules` SQL database table to verify record insertion (`schedule_id`, `target_url`, `cron_expression`, `payload_body`, `is_active = TRUE`).
4. Inspect in-memory `scheduler_mapping` registry for active timer handle.
5. **Restart Recovery Validation:** Force-terminate daemon (`kill -9 <pid>`). Restart daemon. Inspect scheduler initialization logs and `scheduler_mapping`.

- **Expected Result:**
- Timer handle created with unique `schedule_id` and registered in `scheduler_mapping`.
- Database row created in `recurring_schedules` with status `is_active = TRUE`.
- Following daemon restart, startup recovery queries `recurring_schedules`, reconstructs `scheduler_mapping`, recalculates next monotonic execution epoch, and re-arms 100% of active schedules in < 3.0 seconds without client re-registration.

- **Actual Result / Status:** TBD / Not Executed

Test Case ID: TC-FR-06

- **Title:** Verify Precision Timer Callback Execution and Periodic Re-Arming
- **Associated Requirement:** FR-06, NFR-03 (WACDP-6)
- **Test Type:** Functional / Timing Accuracy
- **Priority:** High (P1)
- **Pre-conditions:** Recurring timer active with interval 1,000 ms; mock destination server active.
- **Test Input Data:** Heartbeat task firing every 1,000 ms over 5 cycles.
- **Step-by-Step Execution Procedure:**

1. Register recurring task with 1,000 ms interval.
2. Allow system to run for 5,500 ms (capturing 5 callbacks).
3. Measure timestamps of callback dispatches.

- **Expected Result:**
- Exactly 5 execution events triggered.
- Timing interval between events is 1,000 ms ± 10 ms (NFR-03).
- Timer handle automatically re-arms after each execution.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: TC-FR-07

- **Title:** Verify Dispatcher Thread Network Transmission and HTTP Connection Keep-Alive
- **Associated Requirement:** FR-07, SEC-REQ-03 (WACDP-7)
- **Test Type:** Functional / E2E
- **Priority:** High (P1)
- **Pre-conditions:** Mock HTTP server running on `127.0.0.1:9090` configured to return HTTP 200 OK; TLS 1.3 enabled.
- **Test Input Data:** Prepared dispatch package in `dispatch_queue`.
- **Step-by-Step Execution Procedure:**

1. Push dispatch item into `dispatch_queue`.
2. Dispatcher thread pops item and transmits HTTP POST over TLS 1.3.
3. Inspect received HTTP request at mock server.
4. Observe dispatcher thread state and connection pool.

- **Expected Result:**
- Mock server receives HTTP POST with full payload and header `Connection: keep-alive`.
- Dispatcher marks task `DELIVERED`.
- TCP connection returned to pool for reuse.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: TC-FR-08

- **Title:** Verify Exponential Backoff Retry Loop Timing, Status Code Discrimination, and Max Retries Termination
- **Associated Requirement:** FR-08, NFR-06 (WACDP-7)
- **Test Type:** Functional / Resilience / Timing
- **Priority:** Critical (P0)
- **Pre-conditions:** Mock server configured for programmable error simulation; `MAX_RETRIES = 5`.
- **Test Input Data:**
- Sub-test A (Transient Error): Mock server returns HTTP 503 Service Unavailable.
- Sub-test B (Client Error): Mock server returns HTTP 400 Bad Request.

- **Step-by-Step Execution Procedure:**

1. Execute Sub-test A:

- Transmit initial dispatch attempt (Attempt 0) at T = 0.
- Mock server returns HTTP 503.
- Record timestamps of retries 1, 2, 3, 4, 5.
- Verify delays follow 1s, 2s, 4s, 8s, 16s sequence.
- Verify termination after 1 initial attempt + 5 retries = 6 total attempts.
- Verify task committed to Persistent DLQ and append-only audit log.

2. Execute Sub-test B:

- Transmit initial dispatch attempt (Attempt 0) at T = 0.
- Mock server returns HTTP 400.
- Verify system immediately halts retries and transfers task to DLQ.
- **Expected Result:**
- Sub-test A:
- Attempt 0 (Initial Dispatch) occurs at T ≈ 0s.
- Retry 1 occurs at T ≈ 1s (Δt = 1s = 2^0).
- Retry 2 occurs at T ≈ 3s (Δt = 2s = 2^1).
- Retry 3 occurs at T ≈ 7s (Δt = 4s = 2^2).
- Retry 4 occurs at T ≈ 15s (Δt = 8s = 2^3).
- Retry 5 occurs at T ≈ 31s (Δt = 16s = 2^4).
- System executes exactly 6 total attempts (1 initial + 5 retries) and terminates.
- Task committed to DLQ and append-only audit log.
- Sub-test B:
- HTTP 400 triggers zero retries; task routed immediately to DLQ.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-09`

- **Title:** Verify Persistent Dead Letter Storage (DLQ) and Append-Only Failure Audit Logging
- **Associated Requirement:** `FR-09`, `NFR-04`, `SEC-REQ-04` (WACDP-8, WACDP-11)
- **Test Type:** Functional / Durability
- **Priority:** High (P1)
- **Pre-conditions:** SQLite / relational database initialized; audit log path configured.
- **Test Input Data:** Task exhausted after 6 attempts with terminal error `ECONNREFUSED`.
- **Step-by-Step Execution Procedure:**

1. Hand over exhausted task to Persistent Storage & Logger component.

2. Query `dead_letter_webhooks` table for `task_id`.

3. Inspect append-only audit log file on disk.

- **Expected Result:**
- Record exists in `dead_letter_webhooks` with correct `task_id`, `target_url`, `total_attempts = 6`, and status `'DEAD_LETTER'`.
- JSON entry appended to audit log file.
- Sensitive credentials redacted to `***REDACTED***`.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-10`

- **Title:** Verify Administrative Status Query and System Health Inspection API
- **Associated Requirement:** `FR-10` (WACDP-8)

- **Test Type:** Functional / API
- **Priority:** Medium (P2)
- **Pre-conditions:** Server running with known active tasks and completed DLQ records.
- **Test Input Data:** `task_id` of active and failed webhooks.
- **Step-by-Step Execution Procedure:**

1. Issue `GET /api/v1/webhooks/{task_id}/status`.
2. Issue `GET /api/v1/health`.
3. Issue `GET /api/v1/webhooks/failed`.

- **Expected Result:**
- Status endpoint returns accurate task state (`QUEUED`, `PROCESSING`, `DELIVERED`, `DEAD_LETTER`) in < 20 ms.
- Health endpoint returns system telemetry (queue depths, active worker count, memory usage).
- Failed list returns paginated DLQ entries.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-FR-11`

- **Title:** Verify Dead Letter Inspection and Administrative Task Redrive
- **Associated Requirement:** `FR-11` (WACDP-8)
- **Test Type:** Functional / API / Resilience
- **Priority:** Medium (P2)
- **Pre-conditions:** Failed task residing in DLQ table; target endpoint recovered.
- **Test Input Data:** `POST /api/v1/webhooks/{task_id}/retry`.
- **Step-by-Step Execution Procedure:**

1. Submit redrive request for dead-letter `task_id`.
2. Observe database state change (`redrive_count` incremented).
3. Monitor `service_queue` for task re-injection.
4. Verify task dispatch and successful delivery.

- **Expected Result:**
- API returns HTTP 200 with status `"REQUEUED"`.
- Task successfully re-enters ingestion buffer and dispatches to target.
- DLQ state updated to reflect successful redrive.
- **Actual Result / Status:** TBD / Not Executed

10.2 Non-Functional Test Cases

Test Case ID: `TC-NFR-01`

- **Title:** Validate Ingestion Gateway Response Latency (p99 < 15 ms)
- **Associated Requirement:** `NFR-01` (WACDP-9)
- **Test Type:** Performance / Benchmark
- **Priority:** High (P1)
- **Pre-conditions:** Server running on standard reference hardware; network loopback interface.
- **Test Input Data:** 10,000 synthetic JSON webhook ingestion requests (size: 2 KB each).
- **Step-by-Step Execution Procedure:**

1. Execute load generator issuing 1,000 requests/second continuously for 10 seconds.
2. Aggregate latency metrics across all completed HTTP 202 requests.
3. Calculate p50, p90, p95, and p99 percentiles.

- **Expected Result:**
- 100% of requests return HTTP 202.

- Median latency (p50) ≤ 3 ms.
- 99th percentile (p99) latency is **< 15 ms** (target ≤ 10 ms).
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-02`

- **Title:** Validate High Throughput Sustained Ingestion (5,000 req/sec)
- **Associated Requirement:** `NFR-02` (WACDP-9)
- **Test Type:** Performance / Stress
- **Priority:** High (P1)
- **Pre-conditions:** Multi-threaded load injection harness configured with 100 concurrent HTTP keep-alive connections.
- **Test Input Data:** Continuous stream of valid one-shot webhook requests.
- **Step-by-Step Execution Procedure:**

1. Warm up server for 30 seconds.
2. Ramp up traffic to 5,000 requests/sec.
3. Maintain sustained traffic for 5 minutes (1,500,000 total requests).
4. Monitor CPU utilization, RAM usage, and error rate.

- **Expected Result:**
- Sustained throughput ≥ 5,000 req/sec maintained without dropouts.
- Zero dropped connections or socket reset errors.
- Memory consumption remains stable under 300 MB without leaks.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-03`

- **Title:** Validate Scheduler Timer Precision and Jitter Envelope (± 10 ms)
- **Associated Requirement:** `NFR-03` (WACDP-6)
- **Test Type:** Timing Accuracy / Performance
- **Priority:** High (P1)
- **Pre-conditions:** Monotonic clock scheduler running with 50 active recurring tasks at intervals from 500 ms to 10,000 ms.
- **Test Input Data:** High-frequency timer callback registration.
- **Step-by-Step Execution Procedure:**

1. Register 50 timers with varying millisecond periods.
2. Log actual firing timestamps using `CLOCK_MONOTONIC`.
3. Compute delta |T_actual - T_scheduled| for 1,000 cumulative firing events.

- **Expected Result:**
- 100% of timer firings execute within **± 10 ms** of scheduled target.
- Standard deviation of timer jitter is < 3 ms.
- Zero timer callbacks missed or skipped.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-04`

- **Title:** Validate Data Durability & Zero Message Loss Under Unrecoverable Endpoint Failure
- **Associated Requirement:** `NFR-04`, `FR-09` (WACDP-11, WACDP-8)
- **Test Type:** Durability / Fault Injection
- **Priority:** Critical (P0)
- **Pre-conditions:** Downstream endpoint server terminated (hard connection refusal: `ECONNREFUSED`).
- **Test Input Data:** 500 valid webhook tasks submitted to ingestion gateway.

- **Step-by-Step Execution Procedure:**

1. Ingest 500 tasks into system.
2. Allow all tasks to pass through worker pool to dispatcher.
3. Dispatcher attempts connection, fails, executes retry loop, and exhausts retries.
4. Inspect SQLite / persistent dead-letter database table `dead_letter_webhooks`.
5. Inspect audit log file on disk.

- **Expected Result:**
- Exactly 500 records are committed to persistent DLQ table.
- Each record retains original `task_id`, `target_url`, `payload`, failure code `ECONNREFUSED`, and exact attempt count (6).
- Exactly 500 audit log entries written to disk.
- Message loss rate is **0.00%**.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-05`

- **Title:** Validate High Availability (99.95% Availability over 30-Day Window) & Daemon Crash Recovery
- **Associated Requirement:** `NFR-05` (WACDP-4)
- **Test Type:** Reliability / Soak / Recovery
- **Priority:** High (P1)
- **Pre-conditions:** Webhook Aggregator running as a supervised daemon with persistent SQLite/PostgreSQL database containing active DLQ and `recurring_schedules` records.
- **Test Input Data:** Continuous telemetry over 30-day soak window; process termination signal (`kill -9 <pid>`).
- **Step-by-Step Execution Procedure:**

1. Monitor daemon availability telemetry continuously across a sustained 30-day operational observation window.
2. Compute total observed uptime versus downtime: `Availability = (Uptime / Total_Window_Time) * 100%`.
3. Execute simulated crash: inject unhandled process kill (`kill -9 <pid>`) during active ingestion, timer callbacks, and retries.
4. Automatically restart daemon via process supervisor.
5. Measure time elapsed from process launch to readiness on `/api/v1/health`.
6. Inspect `recurring_schedules` recovery: verify all active recurring timers are re-armed automatically.

- **Expected Result:**
- Operational availability meets or exceeds **99.95%** across the 30-day window (total cumulative unplanned downtime ≤ 21.6 minutes per month).
- Service recovers to operational readiness in strictly **< 3.0 seconds**.
- Zero database corruption; 100% of active recurring schedules in `recurring_schedules` are restored and re-armed without client re-registration.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-06`

- **Title:** Validate Exponential Backoff Mathematical Sequence Fidelity and Absolute Bounded Jitter (≤ ± 100 ms)
- **Associated Requirement:** `NFR-06`, `FR-08` (WACDP-7)
- **Test Type:** Resilience / Timing Verification
- **Priority:** High (P1)
- **Pre-conditions:** High-precision network capture harness measuring TCP SYN timestamps.
- **Test Input Data:** Dispatch task with `MAX_RETRIES = 5` (1 initial attempt + 5 retries = 6 total HTTP attempts) targeted at unresponsive endpoint.
- **Step-by-Step Execution Procedure:**

1. Target mock downstream endpoint programmed to drop all packets (`iptables -j DROP`).

2. Capture egress packets on network interface.
3. Record timestamps of initial attempt and all 5 retries.
4. Calculate measured wait intervals: `Δt_actual(n) = t(n) - t(n-1)`.
5. Verify absolute jitter variation: `|Δt_actual(n) - 2^n| ≤ 100 ms`.

- **Expected Result:**
- Attempt 0 (Initial Dispatch): Dispatched immediately at T ≈ 0s.
- Retry 1: Dispatched at Δt = 1.0s ± 100 ms (2^0).
- Retry 2: Dispatched at Δt = 2.0s ± 100 ms (2^1).
- Retry 3: Dispatched at Δt = 4.0s ± 100 ms (2^2).
- Retry 4: Dispatched at Δt = 8.0s ± 100 ms (2^3).
- Retry 5: Dispatched at Δt = 16.0s ± 100 ms (2^4).
- Exactly 6 total HTTP attempts executed; absolute jitter strictly within ≤ **± 100 ms** across all attempts; task routed to DLQ upon terminal exhaustion.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-07`

- **Title:** Validate Memory Bounding (Peak RSS ≤ 500 MB under 50,000 Tasks) and Dynamic Backpressure (50k Default / 100k Max)
- **Associated Requirement:** `NFR-07`, `FR-02` (WACDP-4)
- **Test Type:** Stress / Boundary / Memory Profiling
- **Priority:** High (P1)
- **Pre-conditions:** Ingestion gateway active on 4-core / 8 GB reference server; worker pool paused to buffer incoming tasks.
- **Test Input Data:** 120,000 tasks containing mixed payloads from 1 KB up to maximum allowable 64.0 KB.
- **Step-by-Step Execution Procedure:**

1. Ingest tasks continuously until queue depth reaches 50,000 tasks (including maximum 64 KB payloads).
2. Monitor process Resident Set Size (RSS) via `/proc/[pid]/statm` and OS memory profiler every 1,000 tasks.
3. Verify that payloads > 2 KB are offloaded to disk-backed staging files via `PayloadReference`, maintaining lightweight ~200-byte task descriptors in RAM.
4. Resume worker threads partially to allow dynamic capacity testing up to 100,000 tasks under sustained burst.
5. Verify warning logging at 90% capacity threshold (45,000 / 90,000).
6. Push queue beyond hard capacity limit (100,000 tasks) and observe response codes.

- **Expected Result:**
- Under 50,000 queued tasks (with payloads up to 64 KB), peak process memory consumption remains strictly **RSS ≤ 500 MB**.
- Dynamic capacity safely expands up to 100,000 tasks without memory exhaustion or process OOM crash.
- At 100% hard capacity, incoming requests are rejected with **HTTP 429 Too Many Requests** (`Retry-After: 5`).
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-08`

- **Title:** Validate Outbound Socket Connection and Read Timeouts (5,000 ms)
- **Associated Requirement:** `NFR-08` (WACDP-7)
- **Test Type:** Network / Transport Resilience
- **Priority:** High (P1)
- **Pre-conditions:** Mock endpoint configured to accept TCP connection but withhold HTTP response bytes (tarpitting).
- **Test Input Data:** Standard webhook dispatch payload.
- **Step-by-Step Execution Procedure:**

1. Dispatch payload to tarpitted endpoint.

2. Monitor dispatcher thread execution state.
3. Measure elapsed time before socket is forcefully aborted.

- **Expected Result:**
- Socket read timeout triggers at exactly 5,000 ms ± 150 ms.
- Dispatcher thread does not freeze or block other tasks.
- Connection is cleanly torn down and task enters retry loop.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-09`

- **Title:** Validate Multi-Threaded Concurrency, ThreadSanitizer Verification (Zero Data Races/Deadlocks/Starvation), and Low Contention (< 5%)
- **Associated Requirement:** `NFR-09`, `FR-02` (WACDP-10)
- **Test Type:** Concurrency / ThreadSanitizer
- **Priority:** Critical (P0)
- **Pre-conditions:** C++20 build compiled with `-fsanitize=thread`; worker pool evaluated across 4, 16, 32, and 64 threads.
- **Test Input Data:** 50,000 tasks pushed to `service_queue` concurrently by 20 ingestion threads.
- **Step-by-Step Execution Procedure:**

1. Run concurrent push/pop stress tests across worker pool configurations of 4, 16, 32, and 64 threads.
2. Profile mutex lock wait time versus total thread execution time under high concurrency.
3. Inspect Clang ThreadSanitizer (TSan) diagnostic report for data races, deadlocks, or lock inversions.
4. Verify absence of worker thread starvation over 10,000 task completions.

- **Expected Result:**
- Under ThreadSanitizer, **zero data races, zero deadlocks, and zero worker starvation** SHALL occur.
- Mutex lock contention time represents strictly **< 5%** of total worker thread CPU cycles.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-10`

- **Title:** Validate Subsystem Modularity and Clean C++20 Interface Independence
- **Associated Requirement:** `NFR-10` (WACDP-5)
- **Test Type:** Architecture / Code Quality
- **Priority:** Medium (P2)
- **Pre-conditions:** Source repository checked out with Clang tooling.
- **Test Input Data:** Unit test harnesses mocking out individual subsystem interfaces.
- **Step-by-Step Execution Procedure:**

1. Execute unit test suites for each isolated module (`IngestionGateway`, `ServiceQueue`, `Scheduler`, `Dispatcher`, `DLQStorage`).
2. Measure decoupled unit test coverage across code modules.

- **Expected Result:**
- Every subsystem is independently unit-testable via mock interfaces without spinning up the full server.
- Code coverage exceeds **85%** statement coverage.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-11`

- **Title:** Validate Multi-Platform POSIX Portability (Linux & macOS Darwin)
- **Associated Requirement:** `NFR-11` (WACDP-4)
- **Test Type:** Portability / Multi-Target Build

- **Priority:** Medium (P2)
- **Pre-conditions:** Build environments on Ubuntu 22.04 LTS (x86_64 / arm64) and macOS Darwin (Apple Silicon).
- **Test Input Data:** Identical C++20 source codebase and CMake configuration.
- **Step-by-Step Execution Procedure:**

1. Compile codebase on Ubuntu 22.04 with GCC 12 and Clang 16.
2. Compile codebase on macOS with Apple Clang 15+.
3. Execute automated regression test suite on both platforms.

- **Expected Result:**
- Zero compilation warnings with `-Wall -Wextra -Werror`.
- 100% of functional and security unit tests pass identically on both platforms.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-NFR-12`

- **Title:** Validate Standards Adherence (RFC 7230, RFC 7231, RFC 8446 & WACDP Webhook Signing Specification)
- **Associated Requirement:** `NFR-12` (WACDP-7)
- **Test Type:** Compliance / Protocol Verification
- **Priority:** Medium (P2)
- **Pre-conditions:** Protocol compliance test tool (curl / openssl s_client / Wireshark).
- **Test Input Data:** Diverse HTTP request formats and TLS negotiation parameters.
- **Step-by-Step Execution Procedure:**

1. Verify HTTP/1.1 message formatting strictly conforms to RFC 7230/7231.
2. Verify webhook signature headers adhere strictly to the WACDP Webhook Signing Specification (`t=<Timestamp>,v1=<SignatureHex>`).
3. Verify TLS 1.3 protocol handshake adheres to RFC 8446.

- **Expected Result:**
- All headers, status codes, and transfer formats strictly conform to RFC 7230/7231.
- Webhook signatures format correctly and verify against HMAC-SHA256 test oracles.
- Handshake negotiation strictly conforms to RFC 8446 TLS 1.3.
- **Actual Result / Status:** TBD / Not Executed

10.3 Security Validation Test Cases

Test Case ID: `TC-SEC-01`

- **Title:** Validate HMAC-SHA256 Payload Signature Generation and In-Flight Tamper Rejection
- **Associated Requirement:** `SEC-REQ-01`, `SEC-OBJ-01`
- **Test Type:** Security / Cryptographic Validation
- **Priority:** Critical (P0)
- **Pre-conditions:** Target endpoint registered with shared secret `sec_live_99fbd0a721`.
- **Test Input Data:**
- Payload: `{"event": "subscription.renewed", "account": "ACC-4411", "tier": "enterprise"}`
- **Step-by-Step Execution Procedure:**

1. Submit task and allow dispatcher to transmit payload to mock recipient.
2. Extract `X-Webhook-Signature` header value `sig_received` and `X-Webhook-Timestamp` header `ts_received`.
3. Calculate reference signature:

```
sig_expected = Hex(HMAC-SHA256("sec_live_99fbd0a721", ts_received + "." + payload))
```

4. Assert `sig_received == sig_expected`.

5. Alter payload body by changing `"enterprise"` to `"free"`.
6. Re-evaluate recipient validation logic against received signature.

- **Expected Result:**
- Untampered payload signature matches `sig_expected` identically.
- Recipient validation of altered payload FAILS cryptographic integrity check.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-SEC-02`

- **Title:** Validate Ingestion Token Authentication and Token-Bucket Rate Throttling
- **Associated Requirement:** `SEC-REQ-02`, `SEC-OBJ-02`, `SEC-OBJ-03`
- **Test Type:** Security / Boundary
- **Priority:** High (P1)
- **Pre-conditions:** Rate limit configured to 1,000 requests per minute per API key.
- **Test Input Data:**
- Sub-test A: Request with missing `Authorization` header.
- Sub-test B: Request with forged bearer token `Bearer tok_forged_malicious_666`.
- Sub-test C: 1,100 rapid requests with valid bearer token `Bearer tok_valid_user`.
- **Step-by-Step Execution Procedure:**

1. Execute Sub-test A. Observe response code and latency.
2. Execute Sub-test B. Observe response code and latency.
3. Execute Sub-test C in a tight loop within 30 seconds.

- **Expected Result:**
- Sub-test A returns **HTTP 401 Unauthorized** in < 2 ms.
- Sub-test B returns **HTTP 401 Unauthorized** in < 2 ms.
- In Sub-test C, requests 1 to 1,000 return **HTTP 202 Accepted**; requests 1,001 to 1,100 return **HTTP 429 Too Many Requests** with header `Retry-After: 60`.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: `TC-SEC-03`

- **Title:** Validate Mandatory Transport Layer Security (TLS 1.3 Only Enforcement)
- **Associated Requirement:** `SEC-REQ-03`, `SEC-OBJ-01`
- **Test Type:** Security / Cryptographic Boundary
- **Priority:** High (P1)
- **Pre-conditions:** OpenSSL client test harness configured for protocol negotiation testing.
- **Test Input Data:**
- Connection A: Plaintext unencrypted HTTP to port 8080.
- Connection B: HTTPS connection attempting TLS 1.2 negotiation (`openssl s_client -tls1_2`).
- Connection C: HTTPS connection attempting TLS 1.3 negotiation (`openssl s_client -tls1_3`).
- **Step-by-Step Execution Procedure:**

1. Attempt Connection A. Observe connection handling.
2. Attempt Connection B. Observe TLS handshake.
3. Attempt Connection C. Observe TLS handshake and cipher suite.

- **Expected Result:**
- Connection A is rejected immediately; plaintext traffic is refused.
- Connection B handshake fails with `handshake_failure`; TLS 1.2 or earlier is strictly refused.
- Connection C negotiates successfully utilizing cipher suite `TLS_AES_256_GCM_SHA384`.
- **Actual Result / Status:** TBD / Not Executed

Test Case ID: TC-SEC-04

- **Title:** Validate Append-Only Audit Trail and JSON-Aware Recursive Secret Redaction
- **Associated Requirement:** SEC-REQ-04, SEC-OBJ-02
- **Test Type:** Security / Data Sanitization
- **Priority:** High (P1)
- **Pre-conditions:** Logging and DLQ subsystem configured to output to `/var/log/wacdp/audit.log` and SQLite DLQ database.
- **Test Input Data:** Deeply nested JSON payload containing credentials across objects and arrays:

```
{
  "event": "user.provisioned",
  "user": {
    "username": "operator1",
    "password": "supersecretpassword123",
    "credentials": {
      "access_token": "tok_live_998877",
      "secret": "top_secret_hmac_key"
    }
  },
  "tokens": [
    {"name": "session", "token": "sess_abcdef123456"},
    {"name": "backup", "api_key": "key_backup_987654"}
  ],
  "headers": {
    "authorization": "Bearer admin_super_token"
  },
  "public_data": {
    "account_id": "ACC-554433",
    "status": "ACTIVE"
  }
}
```

- **Step-by-Step Execution Procedure:**

1. Submit task with above nested JSON payload to ingestion endpoint.
2. Force delivery failure to trigger DLQ persistence and audit logging.
3. Inspect raw text of `/var/log/wacdp/audit.log` and SQLite DLQ database record (`payload_body`).
4. Verify that the JSON-aware recursive sanitizer walks all nested objects and array elements.

- **Expected Result:**
- Audit log is opened in append-only mode (`O_APPEND`).
- None of the plaintext secret values (`supersecretpassword123`, `tok_live_998877`, `top_secret_hmac_key`, `sess_abcdef123456`, `key_backup_987654`, `admin_super_token`) appear anywhere in the database or log file.
- Every sensitive field value (`password`, `access_token`, `secret`, `token`, `api_key`, `authorization`) is strictly replaced with `***REDACTED***`.
- Non-sensitive fields (`username`, `account_id`, `status`, `event`) remain intact with valid JSON formatting.
- **Actual Result / Status:** TBD / Not Executed

10.4 End-to-End Resilience Test Case

Test Case ID: TC-E2E-01

- **Title:** Full Lifecycle: Ingestion, Worker Allocation, Intermittent Downstream Failure, Exponential Backoff Recovery, and DLQ Isolation
- **Associated Requirement:** FR-01, FR-03, FR-07, FR-08, FR-09, FR-11
- **Test Type:** End-to-End (E2E) System Integration
- **Priority:** Critical (P0)
- **Pre-conditions:** Full system running; mock downstream server running with programmable behavior.

- **Test Input Data:**
- Task 1 (Transient Failure): Mock server returns 503 on attempts 1-2, then returns 200 on attempt 3.
- Task 2 (Permanent Failure): Mock server returns 500 on all attempts.
- **Step-by-Step Execution Procedure:**

1. Submit Task 1 and Task 2 simultaneously to ingestion endpoint.

2. Verify immediate HTTP 202 responses for both.

3. Observe Task 1:

- Attempt 0 (Initial Dispatch): Fails (HTTP 503) → waits 1s.
- Retry 1: Fails (HTTP 503) → waits 2s.
- Retry 2: Succeeds (HTTP 200) → marked `DELIVERED`.

4. Observe Task 2:

- Attempt 0 (Initial Dispatch) fails (HTTP 500).
- Retries 1, 2, 3, 4, 5 fail (HTTP 500) with delays 1s, 2s, 4s, 8s, 16s.
- Reaches `MAX_RETRIES = 5` (6 total attempts) → marked `FAILED` → stored in DLQ table.

5. Admin queries `/api/v1/webhooks/failed`. Verify Task 2 is listed.

6. Admin restarts mock server to healthy state and triggers `/api/v1/webhooks/{task_2_id}/retry`.

7. Verify Task 2 is successfully re-delivered.

- **Expected Result:**
- Task 1 successfully recovers and delivers via exponential backoff.
- Task 2 cleanly transitions to DLQ after 6 total attempts (1 initial + 5 retries) without system disruption.
- Admin redrive successfully delivers Task 2.
- **Actual Result / Status:** TBD / Not Executed

End of Software Test Plan & Test Cases